

Pre-Search Checklist — Clone Synthesis Tutor

Complete this before writing code. Save your AI conversation as a reference document.

Project: Clone Synthesis Tutor (Montessori-informed) · Prepared by: val · Date: 2026-05-18

Phase 1: Define Your Constraints

1. Domain Selection

- **Which domain: healthcare, insurance, finance, legal, or custom?**

Custom — ed-tech / elementary-math tutoring. Target concept is fraction equivalence (e.g., $1/2 = 2/4$), per the project brief.

- **What specific use cases will you support?**

One scrollytelling-style lesson on fraction equivalence: a full-page narrative (Jupyter-notebook feel) where authored prose flows top-to-bottom and embedded interactive checkpoints anchor the scroll. Recommended 5-beat shape with three **different** manipulative metaphors in order — Beat 1: ***chocolate bar*** intro (break a 4-piece bar along grooves to see $1/2 = 2/4$); Beat 2: chocolate MC check; Beat 3: ***pizza slicing*** exploration (draggable knife re-anchors equivalence via cuts + topping coverage); Beat 4: pizza MC check; Beat 5: ***paper folding*** consolidation (fold a square to overlay $1/2$ and $2/4$ — the student physically proves equivalence) ending in the final 'check for understanding' MC. Plus 1–2 optional open-ended 'what do you notice?' reflection beats. Multi-metaphor sequencing is intentional — it's the **multiple-representations** principle from math-ed research and the core of why Synthesis-style lessons transfer. ***Pedagogical principles guiding every section below*** (woven into Sections 5, 6, 7, 10, 11, 14, 16): (a) **Concrete before abstract** — student manipulates the material before any prose introduces $1/2 = 2/4$ notation; (b) **Control of error in the material** — the geometry itself shows correctness (two chocolate quarters visibly cover a half, paper folds physically overlay) rather than the tutor announcing it; (c) **Three-period lesson** — introduce (chocolate beat 1) → recognize (chocolate check + pizza explore, beats 2–3) → recall (pizza check + paper fold, beats 4–5); (d) **Description, not praise** — Haiku's system prompt forbids 'great job!' and requires observational feedback ('You put two quarter-bars together and they covered the same space as one half-bar'); (e) **Prepared environment** — only the active manipulative's pieces are on screen per beat (no clutter from prior metaphors); (f) **Redirect, never reprimand** — wrong-answer hints point back to the current material, never criticize; (g) **Self-paced reveal** — the scroll waits for completion before revealing the next beat; (h) **Aesthetic minimalism** — no badges, points, confetti, progress bars; calm serif typography and generous whitespace.

- **What are the verification requirements for this domain?**

Math correctness is deterministic and gated server-side: MC bubble answers checked against a preselected key; manipulative end-states compared to expected fraction values. Critically, the manipulatives are built with Montessori-style ***control of error*** — two chocolate quarter-pieces visibly snap to cover the same area as a half-bar; folded paper visibly overlays prior creases — so the student often **sees** the equivalence before any verdict appears, and the verdict only ratifies what the geometry already showed. Open-ended reflection beats are non-blocking.

Pedagogical tone (warm, observational, no praise-bombing) is verified by sample-review of session transcripts during the build week.

- **What data sources will you need access to?**

No external data sources. Inputs are the authored lesson script (JSON: 5 beats, each with prose + manipulative widget + MC keys + hint-seed lines), the live manipulative state (chocolate / pizza / paper), and the student's bubble selections / reflection text. No student PII storage in v1.

2. Scale & Performance

- **Expected query volume?**

Prototype-scale: 1 student per session, demo runs only during build week + final demo. Inferred: <50 total lesson runs across development and the demo video.

- **Acceptable latency for responses?**

Inferred budgets: narrator TTS first-audio-out p95 < 800ms after a beat reveals (pre-generate paragraph audio one beat ahead to mask this); manipulative animations 60fps (<16ms frame budget — important for the chocolate-snap, pizza-cut, and paper-fold animations to feel tactile); LangGraph node transitions < 500ms; Haiku hint generation after a wrong MC p95 < 1.2s (show a 'thinking...' state if exceeded). The whole feel must read as 'instant scroll-and-respond.'

- **Concurrent user requirements?**

Single user (one student on one iPad). No concurrency requirements; the provided iPad is the only target device for the demo.

- **Cost constraints for LLM calls?**

Not a constraint at prototype scale. Inferred per-lesson cost: ElevenLabs TTS-only (no Conversational AI / no STT) ~\$0.18/1k characters × ~5,000 chars per lesson ≈ \$0.90 voice; Haiku 4.5 across ~10k tokens of paraphrase + hints + reflection reactions at \$1/\$5 per Mtok ≈ <\$0.05. Roughly \$1 per full lesson run.

3. Reliability Requirements

- **What's the cost of a wrong answer in your domain?**

Demo-level: low absolute cost (no real student outcomes at stake), but high reputational cost — a tutor that affirms an incorrect MC bubble or whose narrator sounds robotic kills the 'feels like exploration, not homework' goal from the brief. Treat tutor errors as demo-blocking bugs.

- **What verification is non-negotiable?**

Deterministic check of MC bubble selections and manipulative end-states against the authored key, before any narrator response or next-beat reveal. Haiku's paraphrased reaction / hint never overrides the deterministic verdict. Additionally, the manipulatives themselves are non-negotiably self-correcting (Montessori control-of-error): chocolate pieces snap precisely or not at all; pizza slices have exact angular bounds; paper folds align on the underlying grid.

- **Human-in-the-loop requirements?**

Not in the runtime loop — the lesson must run unattended on the iPad. In the build loop: Patrick (technical contact) reviews session transcripts mid-week to flag pedagogical missteps before final demo.

- **Audit/compliance needs?**

Light. Inferred: store local session transcripts (beat order, MC selections, manipulative state diffs, Haiku calls, reflection texts) for the build week to support iteration and the 1–2 min demo video. No formal compliance regime; recommend confirming with Patrick before any actual-child testing.

4. Team & Skill Constraints

- **Familiarity with agent frameworks?**

Not specified — recommend confirming. Inferred from stack choice (LangGraph): solo builder is comfortable enough with LangGraph state-machine patterns to author the lesson as a node graph.

- **Experience with your chosen domain?**

Not specified — recommend confirming. Ed-tech / fractions pedagogy is well-documented (Synthesis Tutor public materials, fraction-bar canonical curricula, Maria Montessori's fraction insets and three-period lesson); plan to lean on the Brainlift research doc cited in the brief plus Montessori math materials for grounding the manipulative design.

- **Comfort with eval/testing frameworks?**

Not specified — recommend confirming. Inferred: comfortable with pytest / vitest; LLM-eval tooling (LangSmith datasets) will likely be net-new and worth a half-day of upfront investment.

Phase 2: Architecture Discovery

5. Agent Framework Selection

- **LangChain vs LangGraph vs CrewAI vs custom?**

LangGraph (user decision). Fit is excellent: scrollytelling beats map 1:1 to LangGraph nodes; edges encode 'next beat depends on MC result / manipulative state / reflection sentiment.' The brief's 'simple branching logic' requirement is exactly LangGraph's sweet spot.

- **Single agent or multi-agent architecture?**

Single agent. One LangGraph instance owns the lesson; the three manipulatives (chocolate, pizza, paper) and the MC bubble component are UI surfaces, not sibling agents. Multi-agent adds coordination cost with no payoff for a 1-week scripted lesson.

- **State management requirements?**

LangGraph state holds: current_beat_id from the authored set [chocolate_intro, chocolate_check, pizza_explore, pizza_check, paper_fold_final], beat_history, lesson_phase mapped to Montessori three-period lesson (period_1_introduce ↔ chocolate_intro; period_2_recognize ↔ chocolate_check + pizza_explore; period_3_recall ↔ pizza_check + paper_fold_final), per-beat completion status, MC selections, manipulative snapshots, reflection-text history, hint-attempt counter per checkpoint, adaptive-pacing flag. Persist to local storage so a page reload resumes mid-lesson cleanly during demo.

- **Tool integration complexity?**

Low. ~7 internal tools (see Section 7). No external APIs beyond ElevenLabs TTS and the Anthropic API. UI ↔ agent communication via WebSocket events carrying state diffs. Inferred frontend stack: React + dnd-kit (excellent iPad touch support, pointer events, accessibility) + Framer Motion (spring physics for chocolate-snap, pizza-cut, paper-fold animations) + a lightweight scroll-snap library or hand-rolled IntersectionObserver for the notebook scroll

behavior. Considered and rejected: react-konva/PixiJS (canvas overhead not justified for ~10 draggable pieces), HTML5 native drag-drop (broken on iOS Safari touch), Yjs (no collaboration requirement).

6. LLM Selection

- **GPT-5 vs Claude vs open source?**

Claude Haiku 4.5 (user decision). Four roles: (1) drive the LangGraph as a narrator persona — pick the next authored beat based on student performance (adaptive pacing, framed as Montessori 'guide observing readiness'); (2) generate fresh hint copy when a student picks a wrong MC bubble, referencing what they actually did with the current manipulative; (3) lightly paraphrase narrator prose across sessions so two runs don't read identically; (4) react in-character to open-ended reflection text. All four operate over an authored beat set, keeping the brief's 'scripted dialogue' contract intact. ****Prompt discipline — Montessori observational tone****: Haiku's system prompt explicitly forbids generic praise ('great job!', 'awesome!', 'perfect!') and requires **descriptive** feedback that names what the student did with the material: 'You stacked two quarter-pieces of chocolate and they covered exactly the same area as one half-piece.' On wrong answers, the hint must redirect to the material ('Look at the chocolate bar — what if you put two quarter-pieces together?') and never reprimand. This is a system-prompt-level rule, enforced via examples in-context and verified on a labeled rubric set in eval (Section 9).

- **Function calling support requirements?**

Required and central. Haiku must reliably select among ~7 tool calls (`advance_to_beat`, `generate_hint`, `paraphrase_paragraph`, `classify_reflection`, `validate_mc`, `validate_manipulative`, `read_manipulative_state`). No parallel tool use or deep chaining required.

- **Context window needs?**

Small. Per-call context is the current beat, last 2–3 beats of student activity, the relevant manipulative snapshot — well under 10k tokens. Haiku's 200k window is overkill, which is fine.

- **Cost per query acceptable?**

Yes — Haiku 4.5 at ~\$1/\$5 per Mtok puts a full lesson well under \$0.05 in LLM costs. ElevenLabs TTS dominates the per-lesson bill; LLM cost is not a design constraint.

7. Tool Design

- **What tools does your agent need?**

`validate_mc(beat_id, selected_option)` → deterministic correct/incorrect against the authored key. `read_manipulative_state(manipulative_id)` → current state of the active manipulative (chocolate bar pieces / pizza slices + toppings / paper fold creases). `validate_manipulative(beat_id)` → deterministic check the student's end-state matches the expected fraction relation. `generate_hint(beat_id, attempt_count, manipulative_snapshot)` → Haiku hint that names the **specific** manipulative on screen and redirects the student back to it (Montessori 'redirect, never reprimand'). `paraphrase_paragraph(beat_id)` → small variation on authored narrator prose, observational tone enforced by system prompt. `classify_reflection(text)` → categorizes a student's observation (on-topic / partial / off-topic); reaction is always observational, never evaluative. `advance_to_beat(next_beat_id)` → state transition, optionally chosen by Haiku for adaptive pacing.

- **External API dependencies?**

ElevenLabs TTS (text-to-speech only; no Conversational AI, no STT) and Anthropic API (Haiku 4.5). Pre-generate the next beat's audio one beat ahead to hide TTS latency. Three manipulatives (chocolate, pizza, paper) are rendered as separate React components — each uses dnd-kit for drag input and Framer Motion for snap / cut / fold animations. Per **Montessori prepared environment**, the 'tray' on screen reshapes per beat: chocolate beats show only chocolate pieces, pizza beats only the pizza + knife + toppings, paper beat only the square + fold guides — no clutter from prior manipulatives.

- **Mock vs real data for development?**

Not applicable in the usual sense — there is no production data source. Author the lesson script JSON (5 beats — chocolate_intro, chocolate_check, pizza_explore, pizza_check, paper_fold_final — each with prose + manipulative widget config + MC keys + hint-seed lines) on day 1; build a labeled set of ~20 reflection-text examples per category (on-topic / partial / off-topic) to drive classify_reflection eval. Also build a 'hint-tone' rubric set (good vs bad hints) where 'bad' includes praise-bombing ('Great job!') and 'good' is observational + redirects to the active manipulative — this enforces the Montessori prompt discipline in CI.

- **Error handling per tool?**

validate_mc and validate_manipulative are pure-function — cannot fail. generate_hint / paraphrase_paragraph timeout → fall back to canonical authored hint / paragraph text; lesson remains playable. classify_reflection failure → drop the reflection beat to a generic *observational* fallback ('I love that you're noticing things — what did you see in the chocolate?') and continue. ElevenLabs TTS failure → silently degrade to text-only (the paragraph is on the page anyway); never block the lesson.

8. Observability Strategy

- **LangSmith vs Braintrust vs other?**

LangSmith. Native LangGraph integration means zero-config traces of every beat transition and tool call — invaluable for debugging the scroll-and-checkpoint flow during the build week. (In-scope per user decision.)

- **What metrics matter most?**

Beat-completion rate (do students reach the end?), MC accuracy per checkpoint, time-per-beat (where do students stall?), hint-trigger rate per manipulative (chocolate / pizza / paper — which is hardest?), reflection-beat opt-in rate, total LLM + TTS cost per lesson. Also: hint-tone rubric pass rate (does generate_hint comply with Montessori observational discipline?).

- **Real-time monitoring needs?**

Not strictly real-time. LangSmith live-trace view during development is enough; no on-call paging for a prototype.

- **Cost tracking requirements?**

Yes — Haiku token counts and ElevenLabs character counts per session, surfaced in a simple LangSmith dashboard.

9. Eval Approach

- **How will you measure correctness?**

Two-tier eval (in-scope per user decision). (1) Automated: MC validation is trivial (deterministic). `classify_reflection` scored against ~60 labeled reflection examples (3 categories × ~20). `generate_hint` scored on the Montessori hint-tone rubric (1–5 scale: redirects to material? observational? avoids praise-bombing?) using an LLM-as-judge (Sonnet) against ~30 hand-written good/bad hint pairs per manipulative. (2) Human: Patrick (or another reviewer) scores 5 full session transcripts on a rubric — warmth, observational-not-praise tone, smoothness of scroll-and-reveal flow, in-character quality of reflection reactions.

- **Ground truth data sources?**

Hand-authored labeled reflection set + hand-written 'good hint vs bad hint' pairs (the 'bad' set deliberately includes praise-bombing and away-from-material patterns so the eval catches Montessori-discipline regressions). Both built on build-day 1.

- **Automated vs human evaluation?**

Both. Automated gates per-PR changes to prompts/script; human rubric runs once mid-week and once before the final demo recording.

- **CI integration for eval runs?**

Yes. GitHub Actions runs the LangSmith dataset eval on every PR; merge blocked if `classify_reflection` accuracy drops below 85%, if hint-tone rubric falls below 4.0/5 average, or if any high-stakes case (MC incorrect → 'great job!' bug) regresses at all.

10. Verification Design

- **What claims must be verified?**

Every 'correct' verdict shown to the student must come from `validate_mc` or `validate_manipulative` — both deterministic. Haiku-paraphrased reactions and hints vary wording but never override the verdict. Reflection-beat reactions never gate progression and never assert a correctness claim about math.

- **Fact-checking data sources?**

The authored lesson script (MC keys + expected manipulative end-states per beat: chocolate-bar piece positions, pizza-slice angular bounds, paper-fold crease alignment). No external fact-checking; math is closed-form and answers are preselected.

- **Confidence thresholds?**

`classify_reflection` confidence < 0.6 → drop the reflection beat into a safe observational canonical line ('I love that you're noticing — what did you see in the [chocolate / pizza / paper]?') rather than risk an off-tone reaction. MC and manipulative checks have no confidence dimension.

- **Escalation triggers?**

Three consecutive wrong MC attempts on the same checkpoint → route to a 'let's work it out together' authored beat where the narrator demonstrates with the active manipulative — the material teaches, not the narrator (Montessori 'show, don't tell'). Adaptive-pacing flag set to 'struggling' so subsequent beats favor shorter prose chunks and earlier hint surfacing. The hint always points the student *back to the current manipulative*, never away from it.

Phase 3: Post-Stack Refinement

11. Failure Mode Analysis

- **What happens when tools fail?**

ElevenLabs TTS outage → narrator silently switches to text-on-page only; the lesson stays fully playable because the prose is already rendered. Anthropic API outage → all Haiku-generated copy (hints, paraphrase, reflection reactions) falls back to the authored canonical text. `validate_mc` and `validate_manipulative` are pure-function and cannot fail.

- **How to handle ambiguous queries?**

Almost eliminated by design — primary inputs are bubble taps and drag-drop on the current manipulative, both unambiguous. The only ambiguity surface is open-ended reflection text, which routes through `classify_reflection`'s low-confidence canonical fallback (Montessori principle of respect-for-the-learner: never guess at meaning, redirect with an observational invitation).

- **Rate limiting and fallback strategies?**

Demo-scale traffic is well below ElevenLabs and Anthropic free-tier rate limits. Single retry-with-backoff on transient 5xx; surface a brief 'thinking...' UI state if hint generation exceeds 800ms.

- **Graceful degradation approach?**

Every Haiku-generated line has a canonical authored fallback. Every TTS audio file has the on-page paragraph as fallback. The lesson must remain completable end-to-end with both Haiku and ElevenLabs disabled — this is the demo-safety contract.

12. Security Considerations

- **Prompt injection prevention?**

Low risk surface (target user is a 7–10yo, primary inputs are bubble taps and drag-drop on chocolate/pizza/paper). The only injection vector is reflection-text free input — treat as data inside delimited tags in the system prompt; never let it control tool selection.

- **Data leakage risks?**

Reflection text may contain a child's first name or context. Inferred: confirm Anthropic ZDR (zero-data-retention) and ElevenLabs privacy settings before any session involving an actual minor. Recommend confirming with Patrick whether the demo iPad will be used with real children or only adult demoers.

- **API key management?**

Local `.env.local` for `ANTHROPIC_API_KEY` and `ELEVENLABS_API_KEY`; not committed. Production-style secret rotation out of scope for a 1-week prototype.

- **Audit logging requirements?**

Local-only session transcripts (beat order, manipulative state diffs, MC selections, Haiku calls, reflection texts) written to disk per session, for build-week debugging and demo-video review. No external audit pipeline.

13. Testing Strategy

- **Unit tests for tools?**

Yes. `validate_mc` and `validate_manipulative` get the most coverage — every authored beat's correct + incorrect cases across all three manipulatives (chocolate piece combinations, pizza slice configurations, paper fold alignments). `paraphrase_paragraph` and `generate_hint` have snapshot tests (do they call Haiku with the expected prompt structure and Montessori-discipline system prompt?), not output-equality tests.

- **Integration tests for agent flows?**

Yes. Golden-path test (student answers correctly across all 5 beats), plus four failure paths: (a) two wrong chocolate MC attempts then correct, (b) pizza manipulative end-state wrong → hint redirects to pizza → correct, (c) Haiku timeout → canonical hint surfaces, (d) ElevenLabs timeout → text-only mode engages cleanly. Runs in CI on every PR.

- **Adversarial testing approach?**

Out of scope per user decision. Surface area is small (taps + drag-drop + short reflection text from a 7–10yo) and the demo doesn't ship to real children. Flag for any future production hardening.

- **Regression testing setup?**

LangSmith dataset eval gated on PR merge (see Section 9); the labeled reflection set and the hint-tone rubric set (including Montessori-discipline negative cases) are replayed against any prompt/script edit.

14. Open Source Planning

- **What will you release?**

Out of scope — internal prototype for the 1-week challenge (SuperBuilders.school). Not planned for open-source release.

- **Licensing considerations?**

Out of scope — see above.

- **Documentation requirements?**

README is a deliverable per the brief — must cover how to run the app and a brief overview of the technical approach: LangGraph beat-node graph, three-manipulative sequence (chocolate → pizza → paper), three-period lesson mapping, ElevenLabs TTS integration, Haiku's four roles, Montessori-discipline prompt design, manipulative architecture with control-of-error, scrollytelling UX pattern.

- **Community engagement plan?**

Out of scope — see above.

15. Deployment & Operations

- **Hosting approach?**

Web app on Vercel (Next.js or Vite + React frontend); LangGraph backend either co-located as Vercel serverless functions or on Render/Modal if cold starts hurt the hint-generation latency budget. iPad accesses via Safari at the preview URL. ElevenLabs TTS calls happen server-side so the API key stays off the client.

- **CI/CD for agent updates?**

GitHub Actions → run unit tests + LangSmith eval → Vercel preview deploy on PR → manual promote to main demo URL. No canary; no automated rollout staging.

- **Monitoring and alerting?**

LangSmith for trace/cost/quality; Vercel built-in metrics for HTTP errors. No paging — builder watches the LangSmith dashboard during demo runs.

- **Rollback strategy?**

Git revert + Vercel redeploy. Pin a 'last-known-good' tag before the final demo recording so any rollback is one click.

16. Iteration Planning

- **How will you collect user feedback?**

Self + Patrick during the build week; the 1–2 minute demo video is the primary feedback artifact. Inferred: if any children test the demo, run a short post-lesson interview ('which manipulative was the most fun — chocolate, pizza, or paper? where did you get stuck?') alongside session-transcript review.

- **Eval-driven improvement cycle?**

Add any observed misclassified reflection or weak hint to the labeled set immediately; re-run the LangSmith eval (including the Montessori hint-tone rubric) before re-recording the demo video. Tight build-week loop, not a long-term cadence.

- **Feature prioritization approach?**

Brief's three functional requirements (conversational tutor, interactive manipulative, lesson flow with check-for-understanding) instantiated here as a scrollytelling narrative with three sequenced manipulatives (chocolate → pizza → paper) and three MC checks, sequenced via the Montessori three-period lesson. The eight pedagogical principles from Section 1 (concrete-before-abstract, control-of-error, three-period-lesson, description-not-praise, prepared-environment, redirect-not-reprimand, self-paced-reveal, aesthetic-minimalism) are the design north star — cut anything that contradicts them, even if it would otherwise be a 'cool' feature.

- **Long-term maintenance plan?**

Out of scope — 1-week sprint deliverable, not a maintained product. Flag for the next conversation if SuperBuilders wants to extend the prototype to a real curriculum (multi-metaphor sequencing scales naturally to more concepts: division, ratios, percentages — each gets its own three-period progression across fresh manipulatives).